



06-DARS · 14 TADAN

Asinxron JavaScript va Fetch API

Event loop, callback, Promise, async/await, fetch bilan API'ga so'rov yuborish, xatolarni boshqarish, parallel so'rovlar va so'rovni bekor qilish.

O'rta · ~120 daqiqa

Veb-ilova doimo "kutadi": serverdan javobni, foydalanuvchi bosishini, faylning o'qilishini, taymerni. Agar har bir kutish butun dasturni to'xtatib qo'ysa, sahifa qotib qolardi. **Asinxron dasturlash** — uzoq davom etadigan amallarni boshlab, natijasini kutayotganda boshqa ishlarni davom ettirish san'ati. Frontend'ni backend bilan bog'lash aynan shu bilimga tayanadi.

Sinxron va asinxron kod

JavaScript

```
console.log('1. Boshlandi');

setTimeout(() => {
  console.log('3. Taymer ishladi (0 ms bo\'lsa ham!)');
}, 0);

console.log('2. Tugadi');

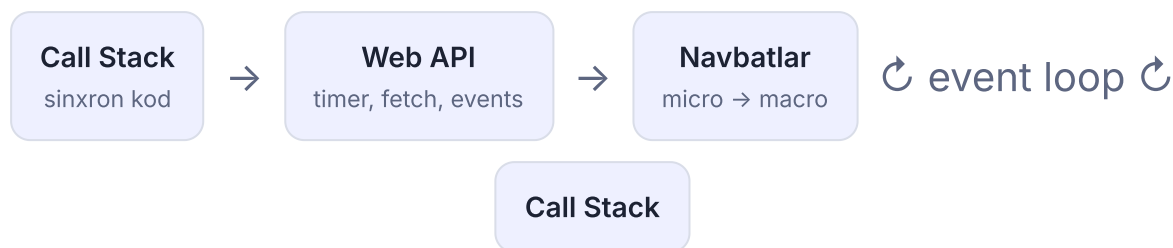
// Natija: 1 → 2 → 3
```

Nega **0 ms** taymer oxirida bajarildi? Javob — **event loop** da.

Event loop — JavaScript qanday ishlaydi

JavaScript **bir oqimli (single-threaded)**: bir vaqtda faqat bitta kod qatorini bajaradi. Lekin brauzer (yoki Node.js) unga yordamchi mexanizmlar beradi:

- **Call Stack (chaqiruvlar steki)** — hozir bajarilayotgan funksiyalar.
- **Web API'lar** — brauzerning o'zi bajaradigan ishlar: taymerlar, tarmoq so'rovlari, hodisalar. Ular JS oqimini band qilmaydi.
- **Macrotask navbati** — `setTimeout`, `setInterval`, hodisa ishlovchilari callback'lari.
- **Microtask navbati** — Promise callback'lari (`.then`, `await` dan keyingi kod). **Ustuvorligi yuqori.**



Stek bo'shagach, avval BARCHA microtask'lar, keyin BITTA macrotask bajariladi, so'ng sahifa qayta chiziladi

Event loop algoritmi: (1) stekdagi sinxron kodni oxirigacha bajar → (2) microtask navbatini to'liq bo'shat → (3) kerak bo'lsa sahifani qayta chiz → (4) macrotask navbatidan bittasini olib, 1-qadamga qayt.

JavaScript

```
console.log('A');
setTimeout(() => console.log('B - macrotask'), 0);
Promise.resolve().then(() => console.log('C - microtask'));
queueMicrotask(() => console.log('D - microtask'));
console.log('E');

// Natija: A, E, C, D, B
```



Event loop'ni bloklamang

Og'ir sinxron hisob (masalan, 5 soniyalik sikl) davomida sahifa butunlay qotadi — tugmalar ishlamaydi, animatsiya to'xtaydi. Node.js serverida esa bunday kod **barcha** foydalanuvchilarning so'rovlarini to'xtatib qo'yadi. Og'ir hisoblar uchun Web Worker (brauzer) yoki worker_threads (Node.js) ishlatiladi.

Callback'lar va "callback hell"

Asinxronlikning eng qadimiy usuli — natija tayyor bo'lganda chaqiriladigan funksiya (callback) berish:

JavaScript

```
function getUser(id, callback) {
  setTimeout(() => callback(null, { id, name: 'Ali' }), 500);
}

// Ketma-ket bog'liq amallar → "piramida" (callback hell)
getUser(1, (err, user) => {
  if (err) return console.error(err);
  getOrders(user.id, (err, orders) => {
    if (err) return console.error(err);
    getOrderDetails(orders[0].id, (err, details) => {
      if (err) return console.error(err);
      getShipping(details.shippingId, (err, shipping) => {
        // ... va hokazo — o'qish, xatolarni boshqarish juda qiyin
      });
    });
  });
});
```

Promise

Promise (va'da) — kelajakda tayyor bo'ladigan natijani ifodalovchi obyekt. U uch holatdan birida bo'ladi:

- **pending** — kutilmoqda;
- **fulfilled** — muvaffaqiyatli bajarildi (qiymat bilan);
- **rejected** — xato bilan tugadi (sabab bilan).

Holat bir marta o'zgaradi va boshqa o'zgarmaydi ("settled").

JavaScript

```
// Promise yaratish
function kutish(ms) {
  return new Promise((resolve) => setTimeout(resolve, ms));
}

function tangaTashlash() {
  return new Promise((resolve, reject) => {
    setTimeout(() => {
      Math.random() > 0.5
        ? resolve('Gerb 🦅')
        : reject(new Error('Raqam chiqdi 🎲'));
    }, 1000);
  });
}

// Promise'ni ishlatish
tangaTashlash()
  .then((natija) => console.log('Yutdingiz:', natija))
  .catch((xato) => console.log('Yutqazdingiz:', xato.message))
  .finally(() => console.log('O\'yin tugadi'));
```

Zanjir (chaining)

`.then()` har doim **yangi Promise** qaytaradi. Callback'dan qaytarilgan qiymat keyingi `.then` ga o'tadi; agar Promise qaytarilsa — u tugashini kutadi. Bu callback piramidasini tekis zanjirga aylantiradi:

JavaScript

```
getUser(1)
  .then(user => getOrders(user.id))
  .then(orders => getOrderDetails(orders[0].id))
  .then(details => getShipping(details.shippingId))
  .then(shipping => console.log(shipping))
  .catch(err => console.error('Zanjirning istalgan joyidagi xato shu yerga keladi:', err));
```

Promise kombinatorlari

Metod	Qachon tugaydi	Natija	Qo'llanilishi
<code>Promise.all</code>	Hammasi bajarilganda yoki birinchi xatoda	Natijalar massivi	Bir-biriga bog'liq bo'lmagan, hammasi kerak bo'lgan so'rovlar
<code>Promise.allSettled</code>	Hammasi tugaganda (xato bo'lsa ham)	<code>{status, value reason}</code> massivi	Ba'zilar xato bo'lsa ham davom etish
<code>Promise.race</code>	Birinchisi tugaganda (muvaffaqiyat yoki xato)	Birinchi natija	Timeout qo'yish
<code>Promise.any</code>	Birinchi muvaffaqiyat lisida	Birinchi muvaffaqiyatli qiymat	Bir nechta zaxira serverdan eng tezi

async / await

`async/await` — Promise ustidagi "sintaktik shakar": asinxron kodni xuddi sinxron koddek yuqoridan pastga o'qiladigan qiladi.

JavaScript

```
async function buyurtmaMalumoti(userId) {
  try {
    const user = await getUser(userId);           // Promise tugashini KUTADI
    (lekin oqimni bloklamaydi)
    const orders = await getOrders(user.id);
    const details = await getOrderDetails(orders[0].id);
    return details;                               // async funksiya DOIM Promise
    qaytaradi
  } catch (err) {
    console.error('Xato:', err.message);
    throw err;                                   // yuqoriga uzatish
  }
}

buyurtmaMalumoti(1).then(d => console.log(d));
// yoki boshqa async funksiya ichida:
// const d = await buyurtmaMalumoti(1);
```

- `await` faqat `async` funksiya ichida (yoki ES modulning yuqori darajasida) ishlatiladi.
- `await` funksiyaning "pauza"ga qo'yadi, lekin event loop boshqa ishlarni davom ettiradi.
- Xatolar oddiy `try...catch` bilan ushlanadi.

Ketma-ket va parallel bajarish

JavaScript

```
// ❌ SEKIN: bir-biriga bog'liq emas, lekin ketma-ket kutilyapti (1s + 1s + 1s = 3s)
const users = await fetchUsers();
const products = await fetchProducts();
const stats = await fetchStats();

// ✅ TEZ: parallel (eng uzuni = 1s)
const [users2, products2, stats2] = await Promise.all([
  fetchUsers(),
  fetchProducts(),
  fetchStats(),
]);

// ⚠️ forEach ichida await ishlamaydi (kutmaydi)!
ids.forEach(async id ⇒ await save(id)); // ❌ — hammasi "uchib ketadi"
for (const id of ids) await save(id); // ✅ ketma-ket
await Promise.all(ids.map(id ⇒ save(id))); // ✅ parallel
```

Fetch API — serverga so'rov yuborish

`fetch()` — brauzerda (va Node.js 18+ da) HTTP so'rov yuborishning standart usuli. U Promise qaytaradi.

JavaScript

```
// GET so'rov
const response = await fetch('https://jsonplaceholder.typicode.com/users');
console.log(response.status); // 200
console.log(response.ok); // true (status 200-299 bo'lsa)
console.log(response.headers.get('content-type'));
const users = await response.json(); // tanani JSON sifatida o'qish (bu ham Promise!)
// boshqa formatlar: response.text(), response.blob(), response.formData()
```

```
// POST so'rov – JSON yuborish
const res = await fetch('https://jsonplaceholder.typicode.com/posts', {
  method: 'POST',
  headers: {
    'Content-Type': 'application/json',
    'Authorization': `Bearer ${token}`,
  },
  body: JSON.stringify({ title: 'Salom', body: 'Matn', userId: 1 }),
});
const created = await res.json();

// PUT / PATCH / DELETE
await fetch(`/api/posts/${id}`, { method: 'PATCH', headers: {...}, body:
JSON.stringify({ title: 'Yangi' }) });
await fetch(`/api/posts/${id}`, { method: 'DELETE' });

// Fayl yuklash – FormData (Content-Type ni o'zingiz qo'ymang!)
const fd = new FormData();
fd.append('avatar', fileInput.files[0]);
await fetch('/api/upload', { method: 'POST', body: fd });

// Query parametrlar
const url = new URL('https://api.example.uz/products');
url.searchParams.set('q', 'telefon');
url.searchParams.set('page', 2);
await fetch(url); // https://api.example.uz/products?q=telefon&page=2
```



Eng ko'p uchraydigan xato

`fetch` faqat **tarmoq xatosida** (internet yo'q, server topilmadi, CORS) reject bo'ladi. Server `404` yoki `500` qaytarsa ham Promise **muvaffaqiyatli** tugaydi! Shuning uchun `response.ok` ni doim tekshiring.

Qayta ishlatiladigan API yordamchisi

api.js

```
export class ApiError extends Error {
  constructor(status, message, data) {
    super(message);
    this.name = 'ApiError';
    this.status = status;
    this.data = data;
  }
}

const BASE_URL = 'http://localhost:3000/api';

export async function api(path, { method = 'GET', body, headers, ...rest } = {}) {
  const token = localStorage.getItem('token');
  const res = await fetch(BASE_URL + path, {
    method,
    headers: {
      ...(body && { 'Content-Type': 'application/json' }),
      ...(token && { Authorization: `Bearer ${token}` }),
      ...headers,
    },
    body: body ? JSON.stringify(body) : undefined,
    ...rest,
  });

  // 204 No Content — tana yo'q
  const data = res.status === 204 ? null : await res.json().catch(() => null);

  if (!res.ok) {
    throw new ApiError(res.status, data?.message || res.statusText, data);
  }
  return data;
}

// Foydalanish:
// const posts = await api('/posts');
// const post = await api('/posts', { method: 'POST', body: { title: 'Salom' } });
```

Bu yordamchini 13-darsdagi full-stack loyihada ishlatamiz.

So'rovni bekor qilish va timeout

JavaScript

```
const controller = new AbortController();

fetch('/api/big-report', { signal: controller.signal })
  .then(r => r.json())
  .catch(err => {
    if (err.name === 'AbortError') console.log('So\rov bekor qilindi');
    else throw err;
  });

controller.abort(); // masalan, foydalanuvchi "Bekor qilish" tugmasini bosdi

// Timeout — zamonaviy qisqa usul
await fetch('/api/slow', { signal: AbortSignal.timeout(5000) }); // 5 soniyadan
keyin bekor
```

CORS haqida qisqacha

Brauzer xavfsizlik uchun **Same-Origin Policy** ni qo'llaydi: `http://localhost:5173` dagi sahifa `http://localhost:3000` dagi API'ning javobini o'qiy olmaydi (port farq qilsa ham — boshqa "origin"). Ruxsat berish **server** zimmasida: u `Access-Control-Allow-Origin` sarlavhasini qaytarishi kerak. Konsolda "blocked by CORS policy" xatosini ko'rsangiz, frontendni emas, backendni sozlang (9-darsda `cors` middleware).

Amaliy misollar

MISOL 1 Foydalanuvchilar katalogi: yuklanish, xato va bo'sh holatlar

Har bir ma'lumot yuklaydigan interfeys **to'rt holatni** ko'rsatishi kerak: yuklanmoqda, xato, bo'sh, ma'lumot.

```

<main>
  <h1>Jamoa a'zolari</h1>
  <button id="reload"><img alt="refresh icon" data-bbox="345 140 365 155"/> Qayta yuklash</button>
  <div id="status" role="status"></div>
  <div id="users" class="grid"></div>
</main>

<style>
  .grid { display: grid; grid-template-columns: repeat(auto-fill,
minmax(240px, 1fr)); gap: 16px; }
  .user { border: 1px solid #e2e8f0; border-radius: 12px; padding: 16px; }
  .user h3 { margin: 0 0 4px; }
  .user small { color: #64748b; }
  .skeleton { height: 110px; border-radius: 12px; background: linear-
gradient(90deg, #f1f5f9 25%, #e2e8f0 50%, #f1f5f9 75%);
background-size: 200% 100%; animation: shimmer 1.2s
infinite; }
  @keyframes shimmer { to { background-position: -200% 0; } }
  .error { color: #b91c1c; background: #fef2f2; padding: 12px; border-
radius: 8px; }
</style>

<script type="module">
  const box = document.querySelector('#users');
  const status = document.querySelector('#status');

  function showSkeleton(n = 6) {
    box.replaceChildren(...Array.from({ length: n }, () => {
      const d = document.createElement('div');
      d.className = 'skeleton';
      return d;
    }));
  }

  function userCard(u) {
    const el = document.createElement('article');
    el.className = 'user';
    const h3 = document.createElement('h3');
    h3.textContent = u.name;
    const email = document.createElement('a');
    email.href = `mailto:${u.email}`;
    email.textContent = u.email;
  }

```

```

    const company = document.createElement('small');
    company.textContent = `${u.company.name} · ${u.address.city}`;
    el.append(h3, email, document.createElement('br'), company);
    return el;
}

async function loadUsers() {
  showSkeleton();
  status.textContent = 'Yuklanmoqda...';
  try {
    const res = await
fetch('https://jsonplaceholder.typicode.com/users', {
  signal: AbortSignal.timeout(8000),
});
    if (!res.ok) throw new Error(`Server xatosi: ${res.status}`);
    const users = await res.json();

    if (users.length === 0) {
      box.replaceChildren();
      status.textContent = 'Hozircha a\'zolar yo\'q.';
      return;
    }
    box.replaceChildren(...users.map(userCard));
    status.textContent = `${users.length} ta a'zo topildi`;
  } catch (err) {
    box.replaceChildren();
    const msg = document.createElement('p');
    msg.className = 'error';
    msg.textContent = err.name === 'TimeoutError'
      ? 'Server javob bermadi. Internetni tekshiring.'
      : `Xatolik: ${err.message}`;
    box.append(msg);
    status.textContent = '';
  }
}

document.querySelector('#reload').addEventListener('click', loadUsers);
loadUsers();
</script>

```

DevTools → Network → **Offline** rejimini yoqib, xato holatini sinab ko'ring. "Slow 3G" bilan skeleton animatsiyasini ko'rasiz.

MISOL 2**Ob-havo ilovasi: debounce qidiruv va ketma-ket so'rovlar**

Open-Meteo — kalitsiz (API key talab qilmaydigan) bepul ob-havo API. Avval shahar nomidan koordinata topamiz (geocoding), keyin shu koordinata bo'yicha ob-havoni olamiz — **ikkinchi so'rov birinchisiga bog'liq**.

```

<div class="weather">
  <input id="city" placeholder="Shahar nomi: Samarqand, Tashkent..."
autocomplete="off">
  <ul id="suggestions"></ul>
  <section id="result"></section>
</div>

<script type="module">
  const input = document.querySelector('#city');
  const list = document.querySelector('#suggestions');
  const result = document.querySelector('#result');

  const debounce = (fn, ms) => { let t; return (...a) => {
clearTimeout(t); t = setTimeout(() => fn(...a), ms); }; };

  let controller; // eski so'rovni bekor qilish uchun

  async function searchCities(query) {
    if (query.length < 2) return list.replaceChildren();
    controller?.abort(); // foydalanuvchi yozishda davom
    etsa — eski so'rov keraksiz
    controller = new AbortController();

    const url = new URL('https://geocoding-api.open-meteo.com/v1/search');
    url.search = new URLSearchParams({ name: query, count: 5, language:
'en' });

    try {
      const res = await fetch(url, { signal: controller.signal });
      const { results = [] } = await res.json();
      list.replaceChildren(...results.map((c) => {
        const li = document.createElement('li');
        li.textContent = `${c.name}, ${c.country}`;
        li.tabIndex = 0;
        li.addEventListener('click', () => showWeather(c));
        return li;
      }));
    } catch (err) {
      if (err.name !== 'AbortError') console.error(err);
    }
  }
}

```

```

const WEATHER_CODES = { 0: '☀️ Ochiq', 1: '☁️ Asosan ochiq', 2: '☀️
Qisman bulutli', 3: '☁️ Bulutli',
  45: '🌫️ Tuman', 61: '🌧️ Yengil yomg\'ir', 63: '🌧️ Yomg\'ir', 71: '🌧️
Qor', 95: '🌨️ Momaqalldiroq' };

async function showWeather(city) {
  list.replaceChildren();
  input.value = city.name;
  result.textContent = 'Yuklanmoqda...';

  const url = new URL('https://api.open-meteo.com/v1/forecast');
  url.search = new URLSearchParams({
    latitude: city.latitude,
    longitude: city.longitude,
    current: 'temperature_2m,wind_speed_10m,weather_code',
    daily: 'temperature_2m_max,temperature_2m_min',
    timezone: 'auto',
  });

  try {
    const res = await fetch(url);
    if (!res.ok) throw new Error(res.statusText);
    const data = await res.json();
    const { temperature_2m: t, wind_speed_10m: w, weather_code: code } =
data.current;

    const days = data.daily.time.map((day, i) =>
      `${new Date(day).toLocaleDateString('uz-UZ', { weekday: 'short',
day: 'numeric' })}: ` +
      `${Math.round(data.daily.temperature_2m_min[i])}°...
${Math.round(data.daily.temperature_2m_max[i])}°`
    );

    result.innerHTML = '';
    const h2 = document.createElement('h2');
    h2.textContent = `${city.name}: ${Math.round(t)}°C
${WEATHER_CODES[code] ?? ''}`;
    const p = document.createElement('p');
    p.textContent = `Shamol: ${w} km/soat`;
    const ul = document.createElement('ul');
    days.forEach((d) => { const li = document.createElement('li');
li.textContent = d; ul.append(li); });
    result.append(h2, p, ul);
  } catch (err) {

```

```
    result.textContent = `Ob-havoni olishda xato: ${err.message}`;  
  }  
}  
  
input.addEventListener('input', debounce((e) =>  
  searchCities(e.target.value.trim()), 400));  
</script>
```

Ikki muhim usul: `debounce` har harf uchun so'rov yubormaslikni ta'minlaydi; `AbortController` esa eski, endi keraksiz so'rovni bekor qiladi — aks holda sekin kelgan eski javob yangi natijaning ustiga yozilib qolishi mumkin ("race condition").

dashboard.js

```
const BASE = 'https://jsonplaceholder.typicode.com';

async function getJSON(path) {
  const res = await fetch(BASE + path);
  if (!res.ok) throw new Error(`${path} → ${res.status}`);
  return res.json();
}

async function loadDashboard() {
  console.time('dashboard');

  // To'rtta mustaqil so'rov PARALLEL; bittasi xato bo'lsa ham qolganlari
  ko'rsatiladi
  const results = await Promise.allSettled([
    getJSON('/users'),
    getJSON('/posts'),
    getJSON('/comments'),
    getJSON('/this-does-not-exist'), // ataylab xato
  ]);

  const [users, posts, comments, broken] = results.map(r =>
    r.status === 'fulfilled' ? r.value : null
  );

  results
    .filter(r => r.status === 'rejected')
    .forEach(r => console.warn('⚠️ Vidget yuklanmadi:',
r.reason.message));

  if (users && posts) {
    // Eng faol 3 ta muallif
    const postCount = posts.reduce((acc, p) => {
      acc[p.userId] = (acc[p.userId] ?? 0) + 1;
      return acc;
    }, {});
    const top = users
      .map(u => ({ ism: u.name, postlar: postCount[u.id] ?? 0 }))
      .sort((a, b) => b.postlar - a.postlar)
```

```

        .slice(0, 3);
        console.table(top);
    }

    console.log({
        foydalanuvchilar: users?.length ?? '-',
        postlar: posts?.length ?? '-',
        izohlar: comments?.length ?? '-',
    });

    console.timeEnd('dashboard'); // ~ eng sekin so'rov vaqti, yig'indisi
    emas
}

loadDashboard();

```

`Promise.all` ishlatilganda bitta buzilgan vidjet butun sahifani "o'ldirardi". `allSettled` esa har bir natijani alohida tekshirishga imkon beradi — real dashboard'larda aynan shunday qilinadi.

MISOL 4

Ishonchli so'rov: timeout, qayta urinish (retry) va eksponensial kutish

Mobil internet uzilib qolishi yoki server vaqtincha band (`503`) bo'lishi mumkin. Professional ilovalar bunday holatlarda so'rovni bir necha marta, har safar uzoqroq kutib, takrorlaydi.

fetchWithRetry.js

```
const sleep = (ms) => new Promise(r => setTimeout(r, ms));

const RETRYABLE = new Set([408, 429, 500, 502, 503, 504]);

export async function fetchWithRetry(url, options = {}, {
  retries = 3,
  timeout = 8000,
  baseDelay = 500,
  onRetry = () => {},
} = {}) {
  for (let attempt = 0; ; attempt++) {
    try {
      const res = await fetch(url, { ...options, signal:
AbortSignal.timeout(timeout) });

      if (res.ok) return res;

      // 4xx xatolarni (404, 401...) takrorlashdan foyda yo'q
      if (!RETRYABLE.has(res.status) || attempt ≥ retries) {
        throw Object.assign(new Error(`HTTP ${res.status}`), { status:
res.status });
      }

      // Server "Retry-After" sarlavhasini yuborgan bo'lsa, unga amal
qilamiz
      const retryAfter = Number(res.headers.get('Retry-After')) * 1000;
      await wait(attempt, retryAfter);
    } catch (err) {
      const network = err.name === 'TypeError' || err.name ===
'TimeoutError';
      if (!network || attempt ≥ retries) throw err;
      await wait(attempt);
    }
  }

  async function wait(attempt, forced) {
    // 500ms, 1000ms, 2000ms ... + tasodifiy "jitter" (hamma bir vaqtda
qayta urinmasligi uchun)
    const delay = forced || baseDelay * 2 ** attempt + Math.random() *
200;
    onRetry({ attempt: attempt + 1, delay: Math.round(delay) });
    await sleep(delay);
  }
}
```

```

}

// Foydalanish
try {
  const res = await fetchWithRetry('https://httpbin.org/status/503', {}, {
    retries: 3,
    onRetry: ({ attempt, delay }) => console.log(`${attempt}-qayta
urinish, ${delay}ms kutamiz...`),
  });
  console.log(await res.text());
} catch (err) {
  console.error('Barcha urinishlar muvaffaqiyatsiz:', err.message);
}

```



Faqat **idempotent** so'rovlarni (GET, PUT, DELETE) avtomatik takrorlang. `POST /payments` ni qayta yuborish ikki marta to'lovga olib kelishi mumkin (1-darsdagi idempotentlik tushunchasini eslang). Bunday holatlar uchun serverlar "Idempotency-Key" sarlavhasidan foydalanadi.

Mashqlar



Mustaqil ish

1. Quyidagi kodning natijasini oldindan ayting, keyin tekshiring:

JavaScript

```

setTimeout(() => console.log(1));
Promise.resolve().then(() => { console.log(2); setTimeout(() =>
console.log(3)); });
(async () => { console.log(4); await null; console.log(5); })();
console.log(6);

```

2. `promiseWithTimeout(promise, ms)` funksiyasini `Promise.race` bilan yozing.

3. GitHub API (`https://api.github.com/users/USERNAME` va `/repos`) orqali profil kartasi yasang: avatar, ism, followerlar va eng ko'p yulduzli 5 ta repo.
4. "Cheksiz scroll": JSONPlaceholder'dan postlarni 10 tadan yuklang (`?_page=2&_limit=10`), sahifa oxiriga yetganda keyingisini (`IntersectionObserver` bilan).
5. 5-darsdagi ro'yxatdan o'tish formasini `https://jsonplaceholder.typicode.com/users` ga POST qiladigan qiling; yuborish davomida tugmani o'chirib, "Yuborilmoqda..." deb ko'rsating.

▼ 1-savol javobi

`4, 6, 2, 5, 1, 3`. Sinxron: 4 va 6. Microtask'lar: 2 va 5. Macrotask'lar: 1, keyin 2-qadamda qo'shilgan 3.

✓ Xulosa

- JavaScript bir oqimli; asinxronlik event loop, Web API'lar va navbatlar orqali ishlaydi. Microtask'lar macrotask'lardan oldin.
- Promise — kelajakdagi natija: pending → fulfilled/rejected.
- `async/await` + `try/catch` — asinxron kodni o'qilishi oson qiladi.
- Mustaqil so'rovlarni `Promise.all` / `allSettled` bilan parallel bajaring.
- `fetch` HTTP xatolarda reject qilmaydi — `response.ok` ni tekshiring.
- `AbortController` — bekor qilish va timeout; retry faqat idempotent so'rovlar uchun.

☐ Darsni o'qib bo'ldim